

Operating System

CS-2006

Lecture 8

Mahzaib Younas

Lecturer Department of Computer Science

FAST NUCES CFD

Outlines

- Overview
- Multicore Programming
- Multithreading Models
- Thread Libraries
- Implicit Threading
- Threading Issues
- Operating System Examples

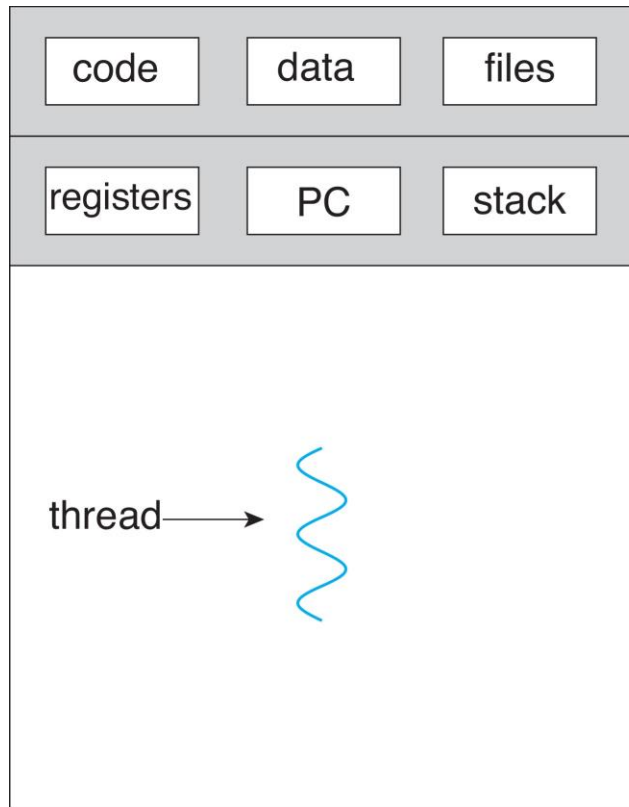
Overview:

- Most modern applications are multithreaded
- Threads run within application
- Multiple tasks with the application can be implemented by separate threads
 - Update display
 - Fetch data
 - Spell checking
 - Answer a network request
- Process creation is heavy-weight while thread creation is light-weight
- Can simplify code, increase efficiency
- Kernels are generally multithreaded

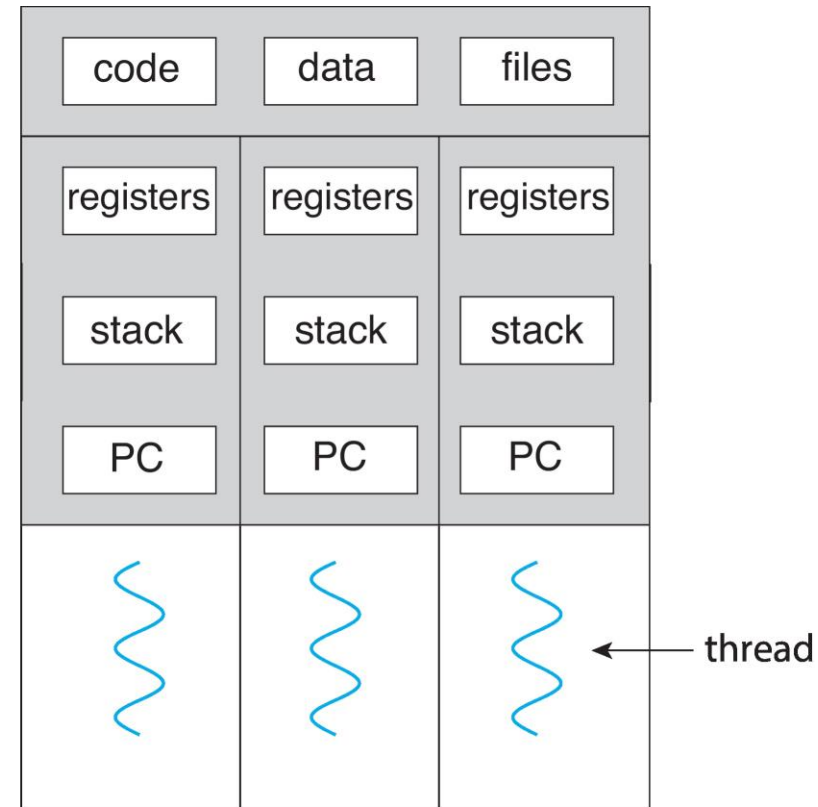
Threading

- A thread is a basic unit of CPU utilization; it comprises
 - thread ID
 - program counter (PC)
 - register set
 - stack.
- It shares with other threads belonging to the same process its
 - code section
 - data section
 - operating-system resources
- For Example: open files and signals.
- A traditional process has a single thread of control. If a process has multiple threads of control, it can perform more than one task at a time.

Threading



single-threaded process

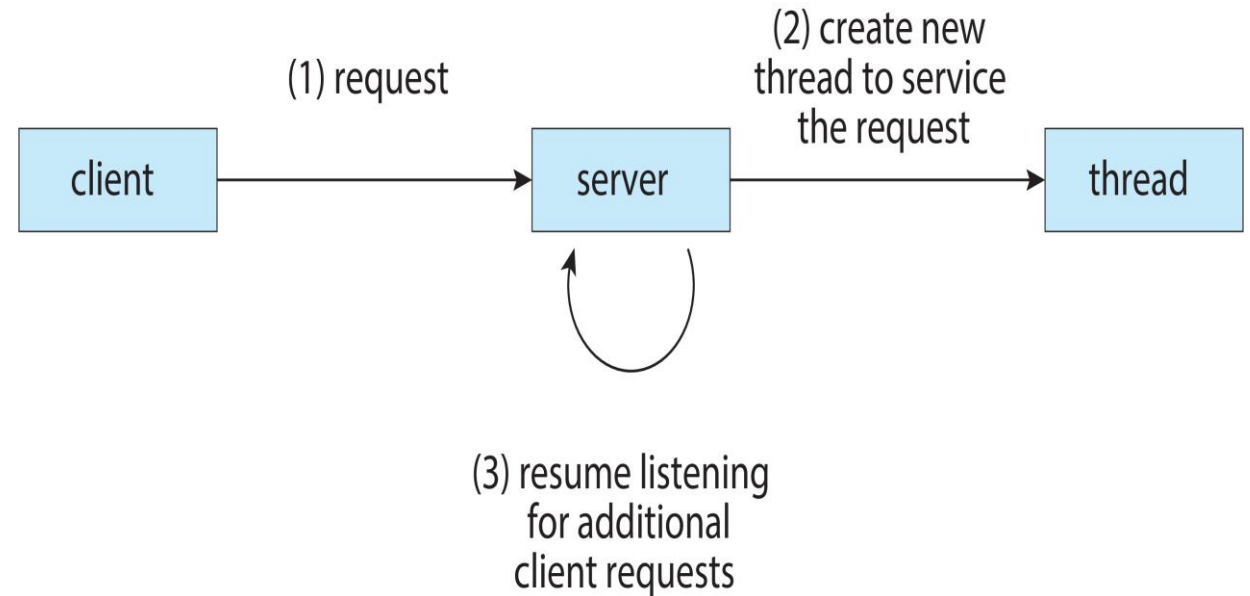


multithreaded process

Multi Threading Server Architecture

Example

If the web-server process is multithreaded the server will create a separate thread that listens for client requests. When a request is made, rather than creating another process, the server creates a new thread to service the request and resumes listening for additional requests



Benefits of Multi Threading

- **Responsiveness**

- may allow continued execution if part of process is blocked, especially important for user interfaces.
- Mouse Click Response

- **Resource Sharing**

- threads share resources of process, easier than shared memory or message passing

- **Economy**

- cheaper than process creation, thread switching lower overhead than context switching

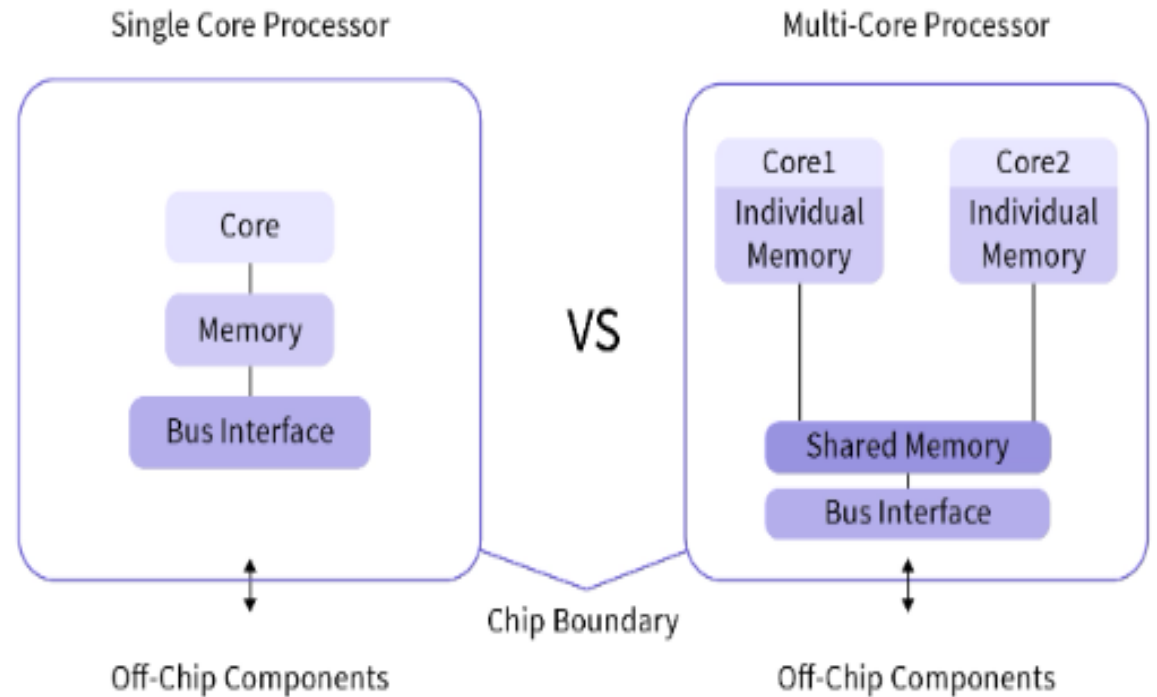
- **Scalability**

- process can take advantage of multicore architectures
- A single-threaded process can run on only one processor, regardless how many are available

Multi Core Programming

A multicore processor system is a single processor with multiple execution cores in one chip.

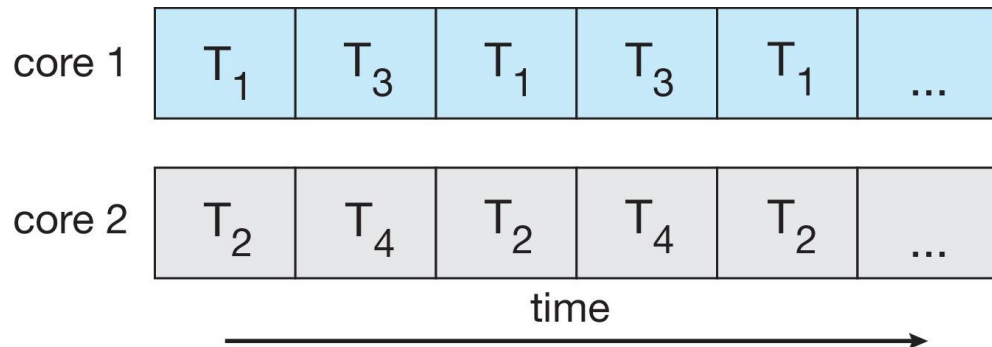
- Multicore or multiprocessor systems puts pressure on programmers, challenges include:
 - Dividing activities
 - Balance
 - Data splitting
 - Data dependency
 - Testing and debugging



Multi Core Programming (Cont..)

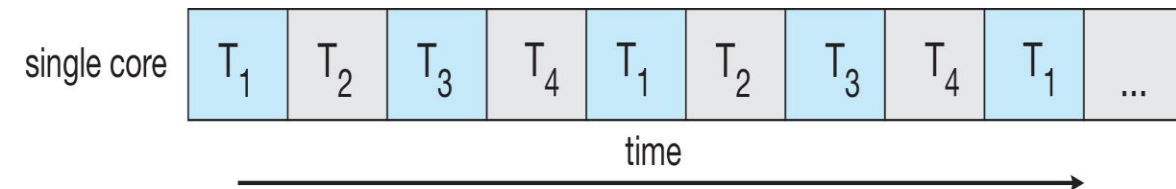
Parallelism

- implies a system can perform more than one task simultaneously
- Execute on multi core system



Concurrency

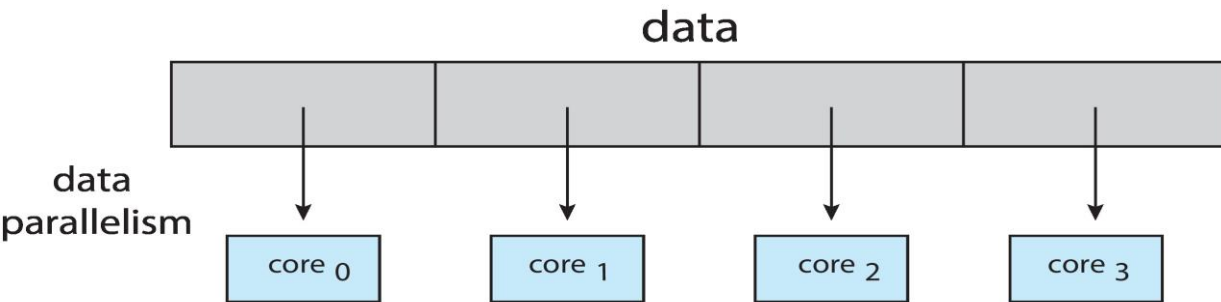
- supports more than one task making progress Single processor / core, scheduler providing concurrency
- Execute on single core System



Multi Core Programming (Cont...)

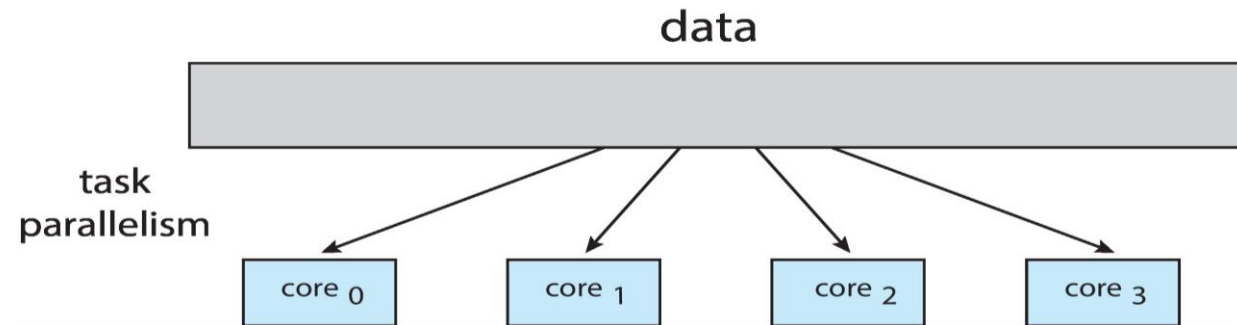
Data Parallelism

distributes subsets of the same data across multiple cores, same operation on each



Task Parallelism

distributing threads across cores, each thread performing unique operation



Amdahl's Law

- Identifies performance gains from adding additional cores to an application that has both serial and parallel components
- S is serial portion
- N processing cores

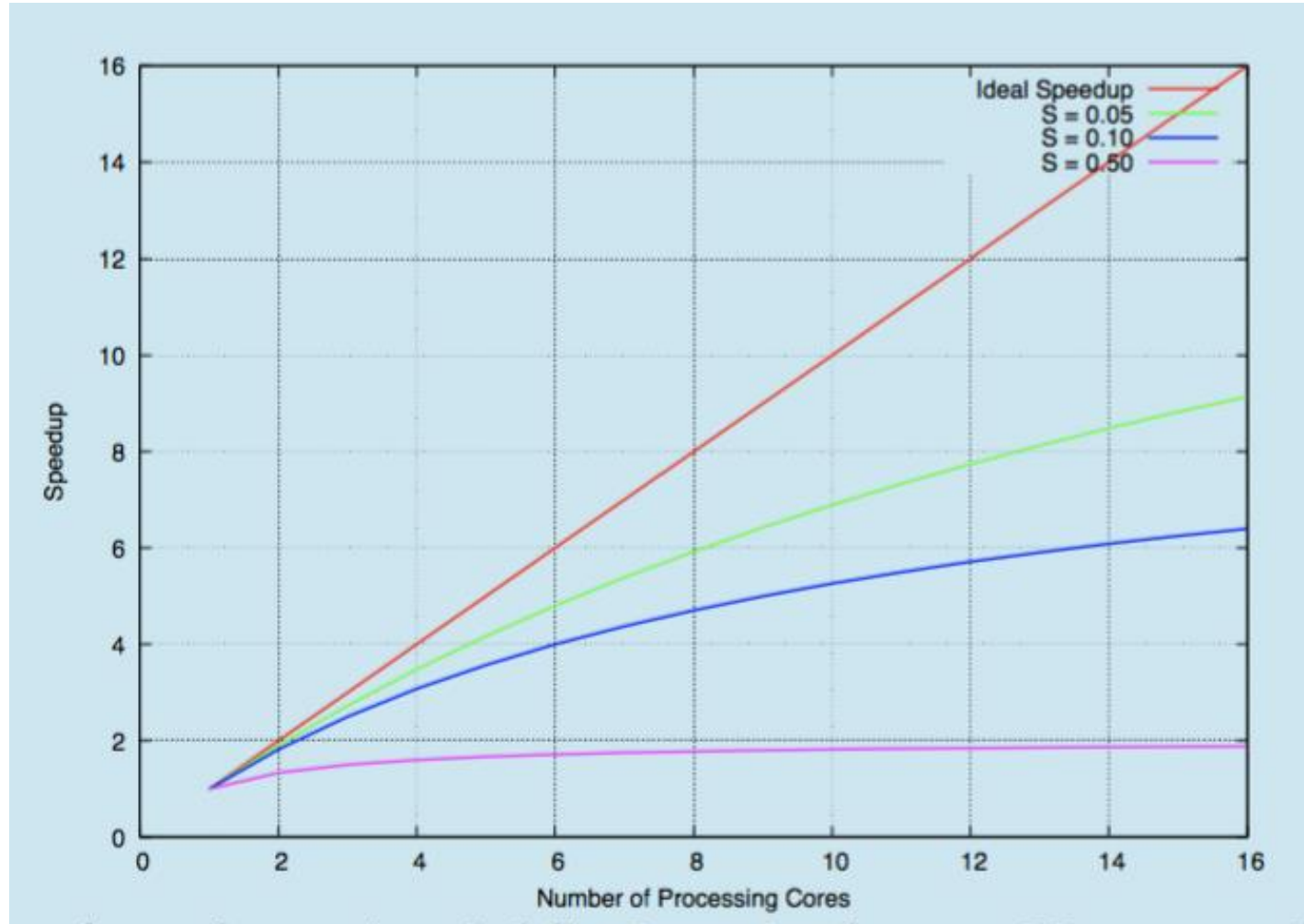
$$speedup \leq \frac{1}{S + \frac{(1-S)}{N}}$$

- That is, if application is 75% parallel / 25% serial, moving from 1 to 2 cores results in speedup of 1.6 times
- As N approaches infinity, speedup approaches $1 / S$

Serial portion of an application has disproportionate effect on performance gained by adding additional cores

- But does the law take into account contemporary multicore systems?

Amdahl Law's (Cont...)



User Threads and Kernel Threads

- **User threads**

- management done by user-level threads library

- Three primary thread libraries:

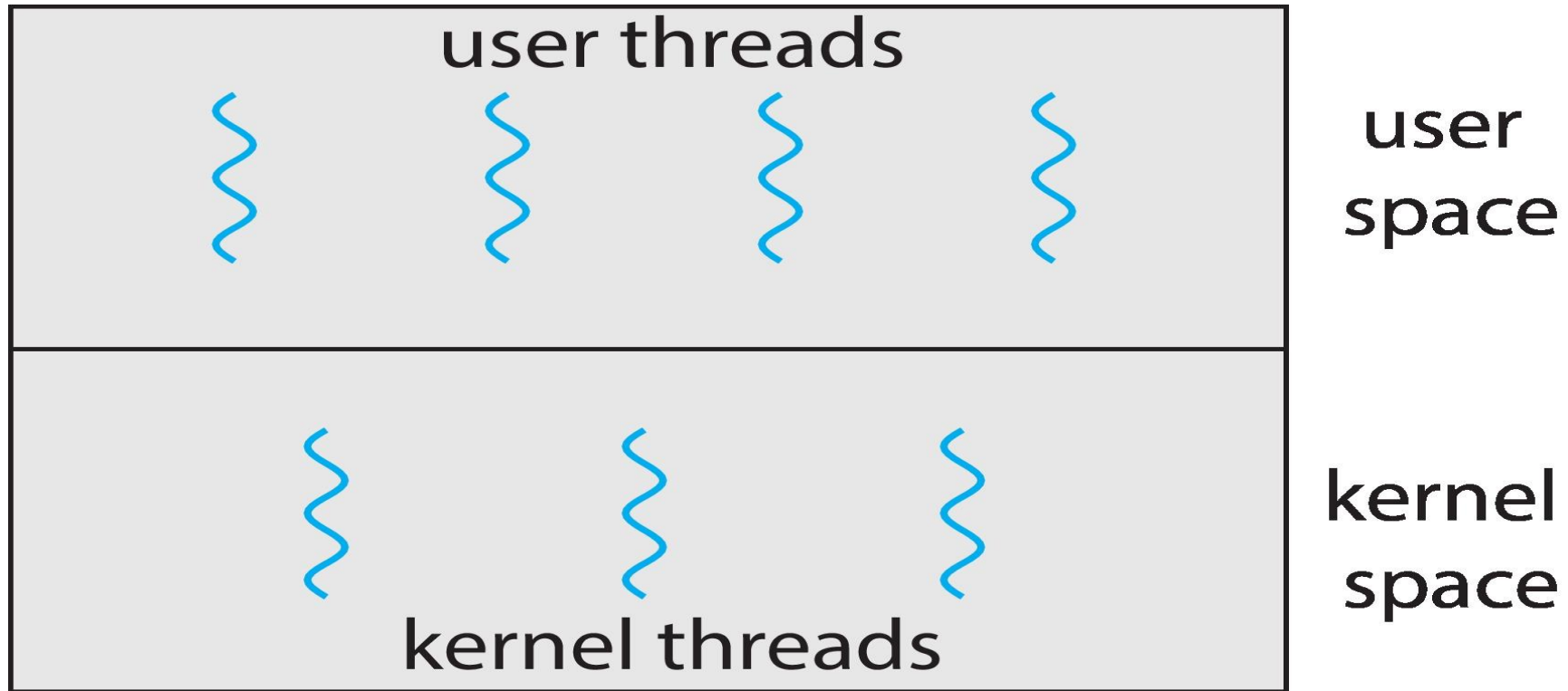
- POSIX **Pthreads**
- Windows threads
- Java threads

- **Kernel Thread:**

Supported by the Kernel

- Examples – virtually all general-purpose operating systems, including:
 - Windows
 - Linux
 - Mac OS X
 - iOS
 - Android

User Threads and Kernel Threads (cont...)



Threading Models

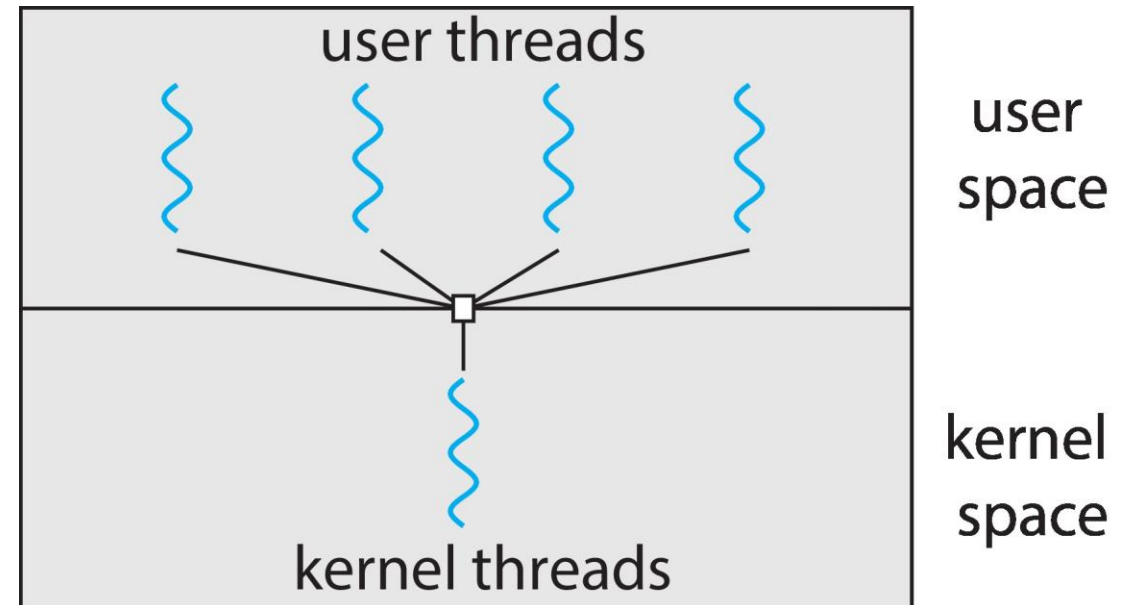
A relationship **must exist between user threads and kernel threads**

- Three common ways of establishing such a relationship:
 - many-to-one model
 - one-to one model
 - many-to-many model

Many to One Model:

Many user-level threads mapped to single kernel thread

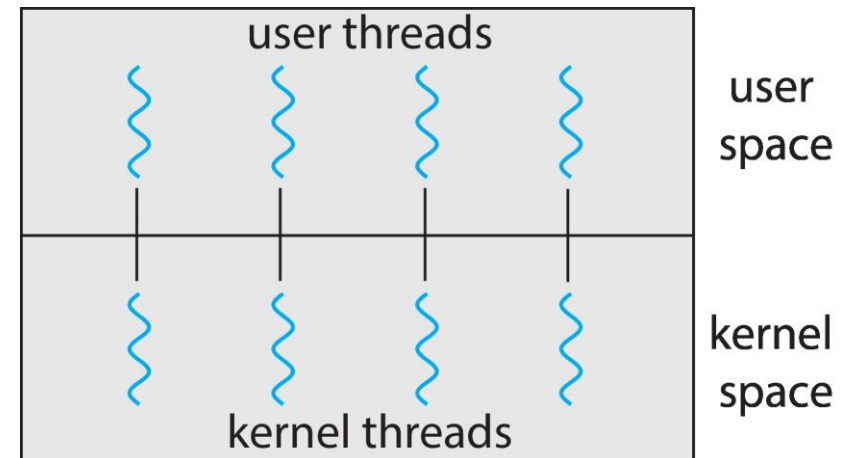
- One thread blocking causes all to block
- Multiple threads may not run in parallel on multicore system because only one may be in kernel at a time
- Few systems currently use this model
- Examples:
 - **Solaris Green Threads**
 - **GNU Portable Threads**



One to One Model

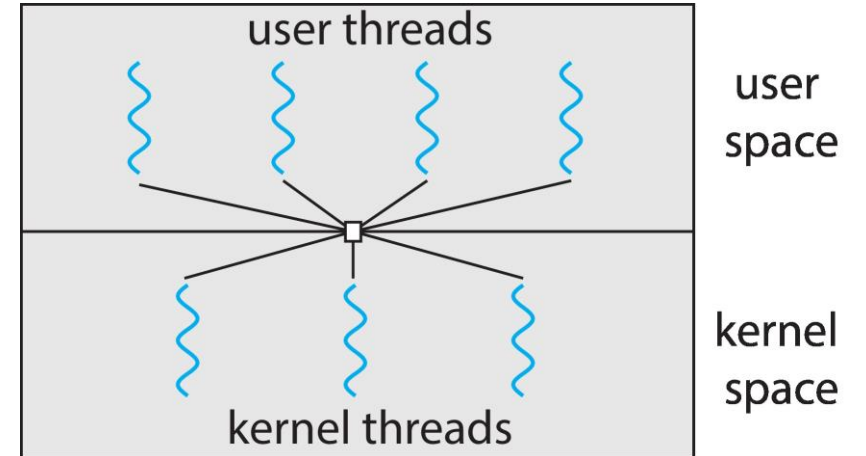
Each user-level thread maps to kernel thread

- Creating a **user-level thread creates a kernel thread**
- **More concurrency** than many-to-one
- Number of threads per process sometimes restricted due to overhead
- Examples
 - Windows
 - Linux



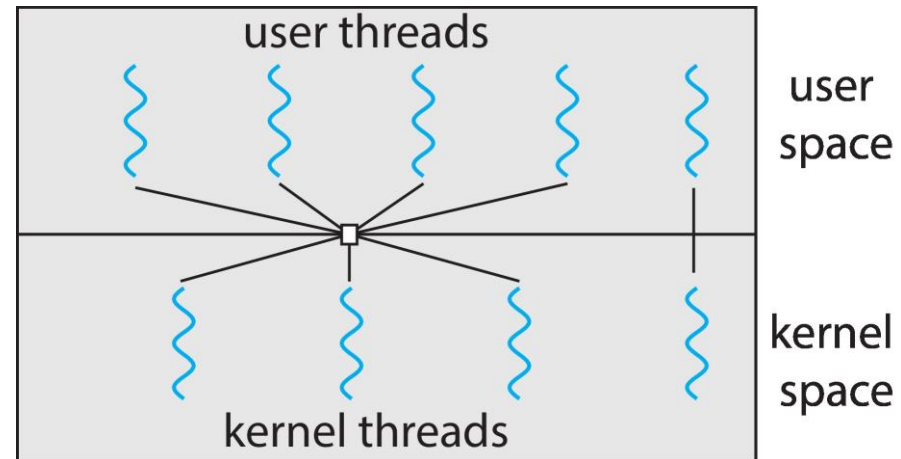
Many to Many Model

- Allows many user level threads to be mapped to many kernel threads
- Allows the operating system to create a sufficient number of kernel threads
- Windows with the *ThreadFiber* package
- Otherwise not very common



Two Mode Model

- Similar to Many to Many
- except that it allows a user thread to be **bound** to kernel thread



Thread Libraries

- Thread library provides **programmer with API for creating and managing threads**
- Two primary ways of implementing
 - Library entirely in user space
 - Kernel-level library supported by the OS

Types of Libraries

- Three main libraries are used
 - POSIX Pthreads
 - The threads extension of the POSIX standard, may be provided as either a user-level or a kernel-level library
 - Windows
 - Windows thread library is a kernel-level library available on Windows systems.
 - Java
 - Java thread API allows threads to be created and managed directly in Java programs.

Pthreads

- A POSIX standard (IEEE 1003.1c) API for thread creation and synchronization
- Specification, not implementation
- API specifies behavior of the thread library, implementation is up to development of the library
- Common in UNIX operating systems (Linux & Mac OS X)

How to create a Thread in C

- **pthread_create**

- It is used to create a new thread.

- Parameters Detail

- Thread

Pointer to an unsigned integer value that return the **thread ID** of the thread that is created.

- Attr

Pointer to a structure that is used to define the **thread attributes** like state, scheduling policy

- Start-Routine

Pointer to subroutine that is **executed** by thread. It return types must be void.

- Args

Pointer to void that contain the arguments to the function which defined earlier

```
int pthread_create(pthread_t * thread,  
                  const pthread_attr_t * attr,  
                  void * (*start_routine)(void *),  
                  void *arg);
```

Pthreads Code Example:

```
#include <pthread.h>
#include <stdio.h>

#include <stdlib.h>

int sum; /* this data is shared by the thread(s) */
void *runner(void *param); /* threads call this function */

int main(int argc, char *argv[])
{
    pthread_t tid; /* the thread identifier */
    pthread_attr_t attr; /* set of thread attributes */

    /* set the default attributes of the thread */
    pthread_attr_init(&attr);
    /* create the thread */
    pthread_create(&tid, &attr, runner, argv[1]);
    /* wait for the thread to exit */
    pthread_join(tid, NULL);

    printf("sum = %d\n", sum);
}
```


Pthread Example

```
/* The thread will execute in this function */  
void *runner(void *param)  
{  
    int i, upper = atoi(param);  
    sum = 0;  
  
    for (i = 1; i <= upper; i++)  
        sum += i;  
  
    pthread_exit(0);  
}
```

Pthreads Code for Joining 10 Threads

```
#define NUM_THREADS 10

/* an array of threads to be joined upon */
pthread_t workers[NUM_THREADS];

for (int i = 0; i < NUM_THREADS; i++)
    pthread_join(workers[i], NULL);
```

Implicit Threading

- Growing in popularity as numbers of threads increase, program correctness more difficult with explicit threads
- Creation and management of threads done by compilers and run-time libraries rather than programmers
- Five methods explored
 - Thread Pools
 - Fork-Join
 - OpenMP
 - Grand Central Dispatch
 - Intel Threading Building Blocks

Thread Pools

- The idea of a thread pool is for the server to create a fixed number of service threads at the time of process start up, and to allow them to 'stay alive' as long as the server operates.
- The service threads that don't have anything to do are kept suspended.
- When a client needs service, and if a service thread is available, the main thread assigns one to the client.
- Otherwise the main thread puts the client in a queue where it waits for a service thread to become available.

Thread Pools (Cont...)

- Create a number of **threads in a pool** where they await work
- Advantages:
- Usually **slightly faster to service a request with an existing thread** than create a new thread
- Allows the **number of threads in the application(s) to be bound** to the size of the pool
- Separating task to be **performed from mechanics of creating task**
allows different strategies for running task
 - i.e., Tasks could be scheduled to run periodically
- Windows API supports thread pools:

Thread Pools (Cont...)

```
DWORD WINAPI PoolFunction(AVOID Param) {  
    /*  
     * this function runs as a separate thread.  
     */  
}
```

DWORD is a 32-bit unsigned integer type defined in Windows API.

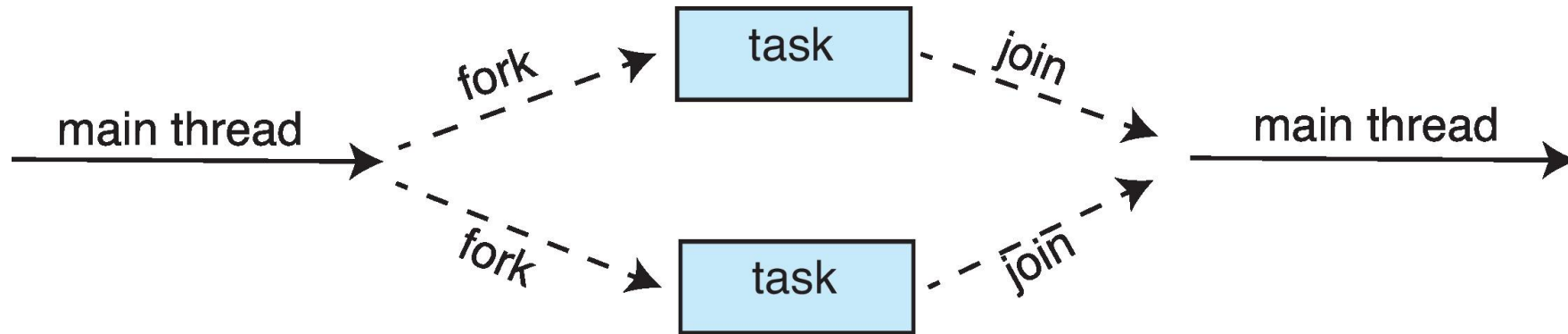
WINAPI is a calling convention used for Windows API functions.

AVOID, LPVOID is a pointer to void, used to pass data to the thread function

Fork Join Parallelism

- version of Java has a **fork-join library** that **spawns threads to perform recursive calls** in algorithms **like Mergesort**.
- Programmers "mark" sections of the source code where parallel work is possible, and the library code takes care of creating and managing threads to do the work

Fork Join Parallelism



Fork–join parallelism delineates a set of tasks that can be executed simultaneously, beginning at the same starting point, the fork, and continuing until all concurrent tasks are finished having reached the join point.

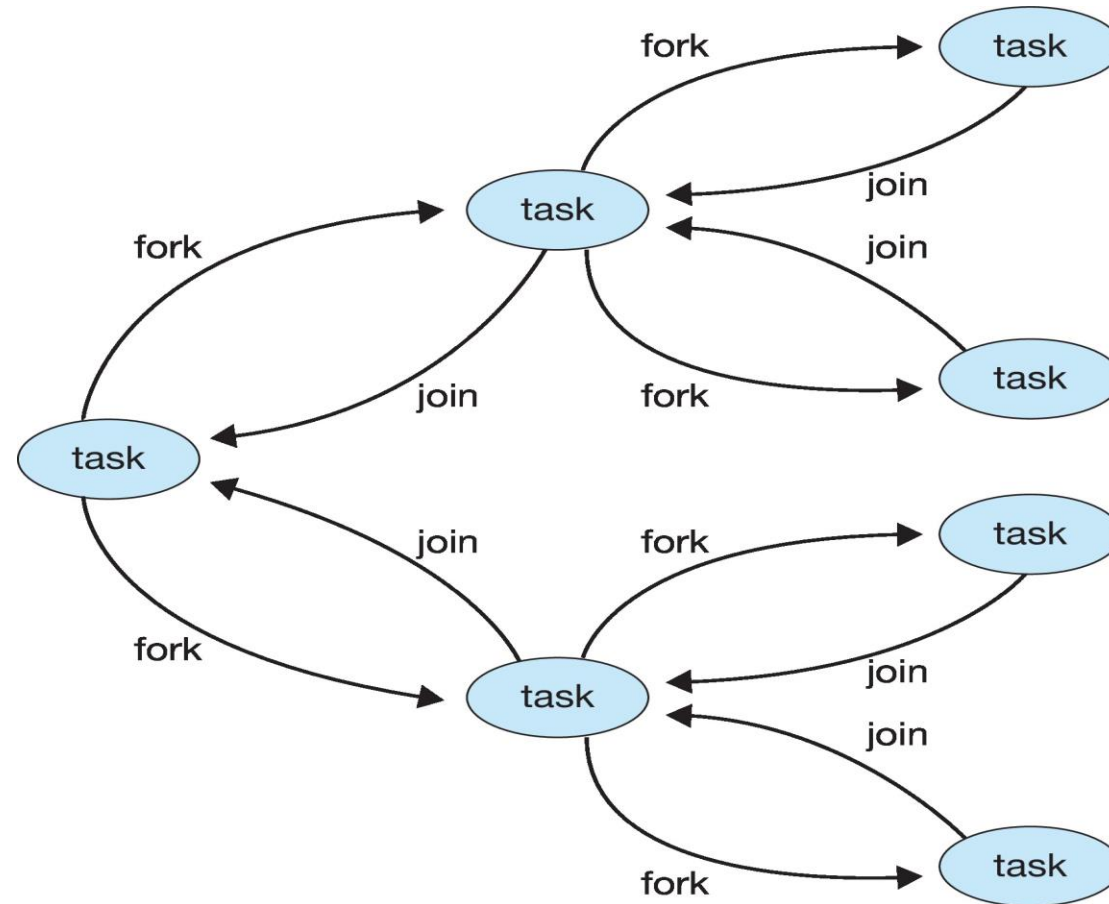
Fork Join Parallelism

```
Task(problem)
  if problem is small enough
    solve the problem directly
  else
    subtask1 = fork(new Task(subset of problem))
    subtask2 = fork(new Task(subset of problem))

    result1 = join(subtask1)
    result2 = join(subtask2)

    return combined results
```

Fork Join Parallelism



Threading Issues

- Semantics of **fork()** and **exec()** system calls
- Signal handling
 - Synchronous and asynchronous
- Thread cancellation of target thread
 - Asynchronous or deferred
- Thread-local storage
- Scheduler Activations

Signal Handling (Cont.)

- Where should a signal be delivered for multi-threaded?
 - Deliver the signal to the thread to which the signal applies
 - Deliver the signal to every thread in the process
 - Deliver the signal to certain threads in the process
 - Assign a specific thread to receive all signals for the process

Thread Cancellation

- Terminating a thread before it has finished
- Thread to be canceled is **target thread**
- Two general approaches:
 - **Asynchronous cancellation** terminates the **target thread immediately**
 - **Deferred cancellation** allows the target thread to **periodically check if it should be cancelled**
- Pthread code to create and cancel a thread:

```
pthread_t tid;

/* create the thread */
pthread_create(&tid, 0, worker, NULL);

. . .

/* cancel the thread */
pthread_cancel(tid);

/* wait for the thread to terminate */
pthread_join(tid, NULL);
```

Thread Cancellation (Cont.)

- Invoking thread cancellation requests cancellation, but **actual cancellation depends on thread state**

Mode	State	Type
Off	Disabled	–
Deferred	Enabled	Deferred
Asynchronous	Enabled	Asynchronous

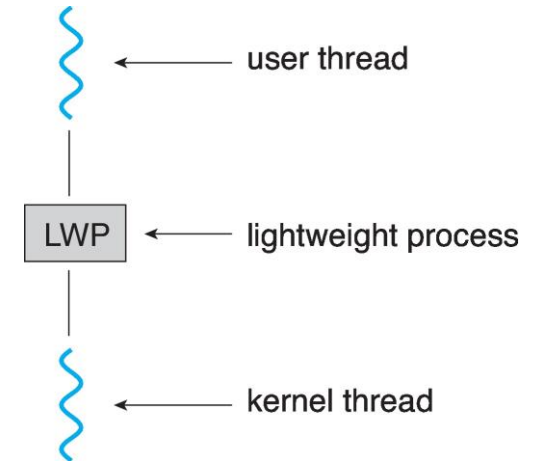
- If thread has **cancellation disabled, cancellation remains pending until** thread enables it
- Default type is **deferred**
 - Cancellation only occurs when thread reaches **cancellation point**
 - i.e., `pthread_testcancel()`
 - Then **cleanup handler** is invoked
- On Linux systems, thread cancellation is handled through signals

Thread-Local Storage

- **Thread-local storage (TLS)** allows each thread to have its own copy of data
- Useful when you do not have control over the **thread creation process** (i.e., **when using a thread pool**)
- Different from local variables
 - Local variables visible **only during single function invocation**
 - TLS visible across function invocations
- Similar to **static** data
 - **TLS is unique to each thread**

Scheduler Activations

- Both **M:M and Two-level models** require communication to maintain the appropriate number of **kernel threads allocated to the application**
- Typically use an intermediate data structure between user and kernel threads – **lightweight process (LWP)**
 - Appears to be a **virtual processor on which process can schedule user thread to run**
 - Each LWP attached to kernel thread
 - How many LWPs to create?
- Scheduler activations provide **upcalls - a communication mechanism from the kernel to the upcall handler in the thread library**
- This communication allows an application to maintain the correct number kernel threads



Linux Threads

- Linux refers to them as **tasks** rather than **threads**
- Thread creation is done through **clone()** system call
- clone() allows a child task to share the address space of the parent task (process)
 - Flags control behavior

flag	meaning
CLONE_FS	File-system information is shared.
CLONE_VM	The same memory space is shared.
CLONE_SIGHAND	Signal handlers are shared.
CLONE_FILES	The set of open files is shared.

- struct task_struct points to process data structures (shared or unique)